

PSEUDOCODE WRITING TASK

CSA07 – Computer Networks | CO5 | Assessment Tool 2

Course Outcome: CO5 – Measure network performance using key metrics with simulation tools. Assessment Tool Weightage: 30%. The uploaded assessment contains three pseudocode-writing problems and a five-part rubric.

[filecite](#) [turn1file0](#) [L2-L11](#)

Purpose of this Work

This 8-page document develops complete, structured pseudocode solutions for all three problems. It follows the assessment wording closely and emphasizes inputs, outputs, validation, loops, conditions, arrays/lists, link-quality scoring, fault handling, alerts and final summaries.

Assessment Questions at a Glance

Question	Main Task	Important Concepts
1	Calculate throughput and packet loss; classify performance	Input validation, arithmetic, IF condition, summary
2	Monitor bandwidth utilization of multiple links	Array/list, loop, threshold condition, backup recommendation
3	Select best path using changing link metrics	Arrays/lists, repeated monitoring, score, path selection, failure handling

Rubric Focus

Criterion	Weight	What the solution must demonstrate
Problem Understanding	20%	All inputs, outputs and constraints identified accurately
Algorithm Design	30%	Complete, correct and logically sound solution
Control Structures	20%	Appropriate loops, conditions and functions/modules
Pseudocode Structure	10%	Clear, consistent and easy-to-follow format
Completeness	20%	All important cases handled with an efficient approach

The rubric gives the greatest weight to algorithm design (30%), while problem understanding and completeness each contribute 20%. [filecite](#) [turn1file0](#) [L46-L57](#)

1. Problem 1 – Throughput and Packet Loss Calculation

The assessment provides 8000 bits received in 4 seconds, with 200 packets sent and 180 packets received. The algorithm must calculate throughput and packet loss, validate time and total packets sent, classify the network as Efficient when throughput > 1500 bps AND packet loss < 10%, and display a complete performance summary.

□ filecite □ turn1file0 □ L12-L22 □

1.1 Inputs, Outputs and Constraints

Item	Details
Input 1	Total data received = 8000 bits
Input 2	Time = 4 seconds
Input 3	Packets sent = 200
Input 4	Packets received = 180
Output 1	Throughput in bits per second
Output 2	Packet loss percentage
Output 3	Network classification: Efficient / Inefficient
Constraint	Time must not be zero; packets sent must not be zero
Classification rule	Throughput > 1500 bps AND packet loss < 10%

1.2 Formulae

- Throughput = Total Data Received / Time
- Packets Lost = Packets Sent – Packets Received
- Packet Loss % = (Packets Lost / Packets Sent) × 100
- Efficient if throughput > 1500 AND packet loss < 10%; otherwise Inefficient.

1.3 Step-by-Step Algorithm

1. Start the algorithm.
2. Read total data received, time, packets sent and packets received.
3. Check whether time is zero. If yes, display an invalid-time message and stop.
4. Check whether packets sent is zero. If yes, display an invalid-packet message and stop.
5. Calculate throughput.
6. Calculate packets lost.
7. Calculate packet loss percentage.
8. Display both calculated metrics.
9. Check the efficiency conditions.
10. Display Efficient if both conditions are satisfied; otherwise display Inefficient.
11. Display a complete network performance summary.
12. Stop.

1.4 Pseudocode – Question 1

```
BEGIN
  INPUT totalDataReceived
  INPUT time
  INPUT packetsSent
  INPUT packetsReceived

  IF time = 0 THEN
    DISPLAY "Invalid input: time cannot be zero."
    STOP
  END IF

  IF packetsSent = 0 THEN
    DISPLAY "Invalid input: total packets sent cannot be zero."
    STOP
  END IF

  throughput ← totalDataReceived / time
  packetsLost ← packetsSent - packetsReceived
  packetLoss ← (packetsLost / packetsSent) × 100

  DISPLAY "Throughput = ", throughput, " bps"
  DISPLAY "Packet Loss = ", packetLoss, "%"

  IF throughput > 1500 AND packetLoss < 10 THEN
    classification ← "Efficient"
  ELSE
    classification ← "Inefficient"
  END IF

  DISPLAY "Network Performance Summary"
  DISPLAY "Total Data Received = ", totalDataReceived, " bits"
  DISPLAY "Time = ", time, " seconds"
  DISPLAY "Packets Sent = ", packetsSent
  DISPLAY "Packets Received = ", packetsReceived
  DISPLAY "Packets Lost = ", packetsLost
  DISPLAY "Throughput = ", throughput, " bps"
  DISPLAY "Packet Loss = ", packetLoss, "%"
  DISPLAY "Classification = ", classification
END
```

1.5 Worked Result Using the Given Values

Metric	Calculation	Result
Throughput	$8000 / 4$	2000 bps
Packets lost	$200 - 180$	20 packets
Packet loss	$(20 / 200) \times 100$	10%
Efficiency test	$2000 > 1500$, but $10\% < 10\%$ is false	Inefficient

Important: the classification uses the exact condition stated in the assessment. Because packet loss must be less than 10% (strictly $< 10\%$), a value of exactly 10% does not satisfy the condition. Therefore, the given data produces an Inefficient classification even though the throughput exceeds 1500 bps.

1.6 Important Points for Marks

- Do not forget input validation before division.
- Use the exact strict comparisons: > 1500 and $< 10\%$.

- Calculate packets lost before calculating packet-loss percentage.
- Display intermediate metrics and the final classification.
- Use clear variable names so the algorithm is easy to understand.

2. Problem 2 – Periodic Bandwidth Utilization Monitoring

The organization has multiple network links and wants to monitor bandwidth utilization and identify overloaded links. The assessment requires periodic collection of bandwidth usage, storage in an array/list, a loop that continuously monitors all links, a condition for utilization above 80%, and a recommendation to switch traffic to a backup link.

□ filecite □ turn1file0 □ L23-L30 □

2.1 Inputs, Outputs and Constraints

Item	Details
Input	Number of network links
Input	Bandwidth utilization for each link
Storage	Array/list for utilization values
Process	Continuously monitor all links
Threshold	Utilization > 80% means overloaded
Output	Link status and backup-switch recommendation
Objective	Efficient bandwidth management and congestion prevention

2.2 Problem Decomposition

13. Identify the links to be monitored.
14. Create a list/array to store utilization values.
15. Periodically collect the current utilization of every link.
16. Check each value against the 80% threshold.
17. Mark overloaded links.
18. Recommend or trigger traffic switching to a backup link.
19. Repeat monitoring after the defined interval.

2.3 Why the Loop is Important

A single measurement cannot show how utilization changes throughout the day. A continuous or periodic loop allows the administrator to detect congestion as it develops. The loop also ensures that every link is checked during each monitoring cycle.

2.4 Pseudocode – Question 2

```
BEGIN
  INPUT numberOfLinks
  CREATE utilization[numberOfLinks]

  WHILE networkMonitoring = TRUE DO

    FOR i ← 1 TO numberOfLinks DO
      utilization[i] ← COLLECT_BANDWIDTH_UTILIZATION(i)

      DISPLAY "Link ", i, " Utilization = ", utilization[i], "%"

      IF utilization[i] > 80 THEN
        DISPLAY "Link ", i, " is OVERLOADED."
        DISPLAY "Recommend switching traffic to backup link."
        SWITCH_OR_RECOMMEND_BACKUP(i)
      ELSE
        DISPLAY "Link ", i, " utilization is acceptable."
      END IF
    END FOR

    WAIT monitoringInterval
  END WHILE
END
```

2.5 How It Prevents Congestion

- Early detection: the 80% threshold identifies a link before utilization becomes extreme.
- Traffic redistribution: overloaded links can move selected traffic to backup capacity.
- Continuous visibility: repeated monitoring reveals changing usage patterns.
- Centralized storage: arrays/lists allow current values to be processed systematically.
- Scalability: the same loop can monitor a larger number of links.

2.6 Improvements for a Stronger Algorithm

For an assessment-level solution, the core requirement is the 80% condition and backup recommendation. A stronger implementation can additionally check whether a backup link is available before switching traffic.

2.7 Control Structures Used

Construct	Use in Algorithm
WHILE loop	Repeats monitoring continuously.
FOR loop	Checks every network link during each cycle.
IF / ELSE	Determines overloaded versus acceptable utilization.
Array/List	Stores current utilization values.
Function/Module	Collects utilization and can perform switching.
WAIT	Creates a regular monitoring interval.

2.8 Example Monitoring Cycle

Link	Utilization	Status	Action
Link 1	65%	Normal	Continue monitoring
Link 2	82%	Overloaded	Recommend/switch to backup
Link 3	74%	Normal	Continue monitoring
Link 4	91%	Overloaded	Recommend/switch to backup

The exact numerical utilization values in this example are illustrative; the assessment itself specifies the decision threshold of greater than 80%. □ filecite□turn1file0□L25-L30□

2.9 Key Explanation for Presentation

The algorithm supports efficient bandwidth management by continuously observing every link and responding when utilization exceeds the defined threshold. Instead of waiting for complete congestion, the administrator can redistribute traffic to backup capacity. This improves link availability and helps reduce the probability of severe congestion.

3. Problem 3 – Best Path Selection Using Link Quality

The third problem involves six branch offices connected through multiple links. Each link has bandwidth, delay and utilization values that change throughout the day. The algorithm must periodically collect these metrics, store them in arrays/lists, calculate a link-quality score, select the path with the highest score, detect overload/failure, select an alternate path and generate an administrator alert. □filecite□turn1file0□L31-L45□

3.1 Inputs, Outputs and Constraints

Item	Requirement
Network	Headquarters connected to six branch offices
Metrics	Bandwidth, delay, utilization
Storage	Suitable arrays/lists
Monitoring	Repeated at regular intervals
Decision	Highest link-quality score
Failure handling	Detect failed/overloaded links
Recovery	Select alternate path
Alert	Notify administrator
Objective	Reliability, routing efficiency and fault tolerance

3.2 Why Shortest Distance Alone Is Not Enough

A physically short path may still be a poor communication path if it has low bandwidth, high delay or high utilization. The assessment specifically asks for overall link quality rather than simply shortest distance. Therefore, the decision must combine the collected performance metrics.

3.3 Link Quality Score – Concept

The assessment requires a score but does not specify a fixed mathematical scoring formula. Therefore, the pseudocode below uses a normalized weighted score as a reasonable algorithm-design choice. Higher bandwidth increases the score, while higher delay and utilization reduce it.

Example conceptual score: $\text{QualityScore} = W_b \times \text{NormalizedBandwidth} + W_d \times (1 - \text{NormalizedDelay}) + W_u \times (1 - \text{NormalizedUtilization})$, where $W_b + W_d + W_u = 1$.

The weights should be selected according to organizational priorities. If the goal is real-time communication, delay may receive greater weight; if capacity is the main concern, bandwidth may receive greater weight.

3.4 Pseudocode – Question 3

```
BEGIN
  INPUT numberOfBranches = 6
  INPUT numberOfPaths

  WHILE monitoring = TRUE DO

    FOR each path p DO
      bandwidth[p] ← COLLECT_BANDWIDTH(p)
      delay[p] ← COLLECT_DELAY(p)
      utilization[p] ← COLLECT_UTILIZATION(p)
      status[p] ← CHECK_LINK_STATUS(p)

      IF status[p] = "FAILED" OR utilization[p] > 80 THEN
        qualityScore[p] ← -1
      ELSE
        normalizedBandwidth ← NORMALIZE_BANDWIDTH(bandwidth[p])
        normalizedDelay ← NORMALIZE_DELAY(delay[p])
        normalizedUtilization ← NORMALIZE_UTILIZATION(utilization[p])

        qualityScore[p] ←
          Wb × normalizedBandwidth +
          Wd × (1 - normalizedDelay) +
          Wu × (1 - normalizedUtilization)
      END IF
    END FOR

    bestPath ← PATH_WITH_HIGHEST_SCORE(qualityScore)

    IF bestPath = NONE THEN
      bestPath ← FIND_ALTERNATE_AVAILABLE_PATH()
      DISPLAY "No preferred path available."
      GENERATE_ALERT("All preferred paths unavailable.")
    ELSE
      DISPLAY "Selected Best Path = ", bestPath
      DISPLAY "Quality Score = ", qualityScore[bestPath]
    END IF

    IF CURRENT_PATH_FAILED(bestPath) = TRUE THEN
      alternatePath ← FIND_ALTERNATE_AVAILABLE_PATH()

      IF alternatePath EXISTS THEN
        SELECT alternatePath
        GENERATE_ALERT("Path failure: traffic moved to alternate path.")
      ELSE
        GENERATE_ALERT("No alternate path available.")
      END IF
    END IF

    WAIT monitoringInterval
  END WHILE
END
```

3.5 Important Design Notes

- Bandwidth should be normalized before combining it with delay and utilization because the units are different.
- Delay and utilization are negative factors, so lower values should produce better contributions to the score.
- A failed path must not be selected as the best path.
- An overloaded path can be excluded or heavily penalized.
- The monitoring loop allows the selected path to change as network conditions change.
- An alert is generated when a failure or serious path problem requires administrator attention.

3.6 How the Algorithm Improves the Network

Objective	How Algorithm Helps
Reliability	Avoids failed/overloaded paths and uses alternatives.
Routing efficiency	Chooses a path using multiple performance metrics.
Fault tolerance	Automatically searches for an alternate path after failure.
Adaptability	Recalculates scores periodically as conditions change.
Monitoring	Maintains current bandwidth, delay and utilization data.
Administration	Generates alerts when important failures occur.

4. Rubric-Based Checklist for All Three Problems

Rubric	How to Score Well
Problem Understanding – 20%	Clearly state every input, output, threshold, validation rule and objective.
Algorithm Design – 30%	Use correct formulas, logical sequence, complete decisions and meaningful variable names.
Control Structures – 20%	Use IF/ELSE for decisions, FOR/WHILE for repeated monitoring, and functions/modules where useful.
Pseudocode Structure – 10%	Use BEGIN/END, indentation, consistent assignment arrows and readable DISPLAY statements.
Completeness – 20%	Handle zero/invalid inputs, overload, failure, missing backup paths and final outputs.

5. Most Important Points to Mention in Viva

20. Throughput measures the amount of data successfully received per unit time.
21. Packet loss percentage shows the proportion of sent packets that were not received.
22. Input validation prevents division-by-zero errors.
23. In Question 1, the network is Efficient only when both conditions are true.
24. An AND condition means failure of either requirement makes the classification Inefficient.
25. In Question 2, utilization greater than 80% triggers overload handling.
26. Arrays/lists store measurements so that multiple links can be processed systematically.
27. A loop enables continuous or periodic network monitoring.
28. Question 3 selects a path using bandwidth, delay and utilization rather than distance alone.
29. Normalization is useful when combining metrics with different units.
30. Failed or overloaded paths should not be selected as the best path.
31. Alternate-path selection improves fault tolerance.
32. Alerts inform the administrator when automated recovery needs attention.
33. Periodic recalculation makes the algorithm adapt to changing network conditions.

6. Final Summary

These three pseudocode algorithms demonstrate constraint-based algorithm design for computer-network performance. Question 1 focuses on calculation, validation and classification; Question 2 focuses on continuous monitoring and congestion prevention; and Question 3 focuses on dynamic path selection, fault tolerance and alerts. Together, they cover the assessment's major requirements: understanding inputs and outputs, correct algorithm design, control structures, clear pseudocode and completeness. □filecite□turn1file0□L57-L129□

Note: Where the assessment specifies a requirement but does not provide a particular implementation formula—especially the exact link-quality score in Question 3—the document identifies the choice as an algorithm-design assumption rather than presenting it as a requirement from the source.